Vertical Federated K-Means for Multi-View Data Guided by a K-Means Cost Bound after Projection

Feijiang Li
Key Laboratory of Evolutionary Science Intelligence of Shanxi Province, Shanxi University, Institute of Big Data Science and Industry
Taiyuan, Shanxi, China
fjli@sxu.edu.cn

Jinhao Jiang
Shanxi University, Institute of Big Data Science and Industry
Taiyuan, Shanxi, China
jhjiang0528@163.com

Jieting Wang
Key Laboratory of Evolutionary Science Intelligence of Shanxi Province, Shanxi University, Institute of Big Data Science and Industry
Taiyuan, Shanxi, China
jtwang@sxu.edu.cn

Liang Du
Key Laboratory of Evolutionary Science Intelligence of Shanxi Province, Shanxi University, Institute of Big Data Science and Industry
Taiyuan, Shanxi, China
duliang@sxu.edu.cn

Yuhua Qian*
Key Laboratory of Evolutionary Science Intelligence of Shanxi Province, Shanxi University, Institute of Big Data Science and Industry
Taiyuan, Shanxi, China
jinchengqyh@126.com

Abstract

Multi-view data is widely present in the real world. Multi-view clustering is an unsupervised method for capturing the grouping structure of such data. However, multi-view clustering struggles to meet the requirements of real-world scenarios, such as distributed storage of different views and data protection needs. These requirements align with the setting of vertical federated clustering. However, vertical federated clustering still faces two challenges: (1) Under the constraints of privacy protection mechanisms, how to theoretically analyze the clustering consistency between the data uploaded by clients to the server and the original client data is challenging. (2) The feature space differences among different clients make cross-view information sharing and fusion difficult. To address the first challenge, we provide a theoretical analysis of the upper bound of the loss of k-means for transformation matrix mapping, revealing the relationship between the k-means loss of the transformed data and the original data. We then propose a vertical federated clustering method (V-HDKM). In this method, clients handle the second challenge by transposing the feature matrix. Guided by the projected k-means loss bound, we expand the feature space and perform k-means clustering to obtain feature cluster centers, which are then uploaded to the server. The server aggregates the global centers and feeds back the optimized results, achieving cross-view knowledge fusion through iterative interactions. Experimental results show that V-HDKM significantly improves local clustering performance and performances better than other seven vertical federated mthods on 20 multi-view datasets. Furthermore, sensitivity analysis on 8 UCI datasets with respect to the number of clients demonstrates the stability of the method. The code is available at https://github.com/jiangjh/V-HDKM.

CCS Concepts

• **Computing methodologies** → **Cluster analysis**; *Unsupervised learning*.

Keywords

Vertical Federated Clustering, Multi-view Federated clustering, K-Means Cost Bound after Projection

ACM Reference Format:
Feijiang Li, Jinhao Jiang, Jieting Wang, Liang Du, and Yuhua Qian. 2026. Vertical Federated K-Means for Multi-View Data Guided by a K-Means Cost Bound after Projection. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780182

1 Introduction

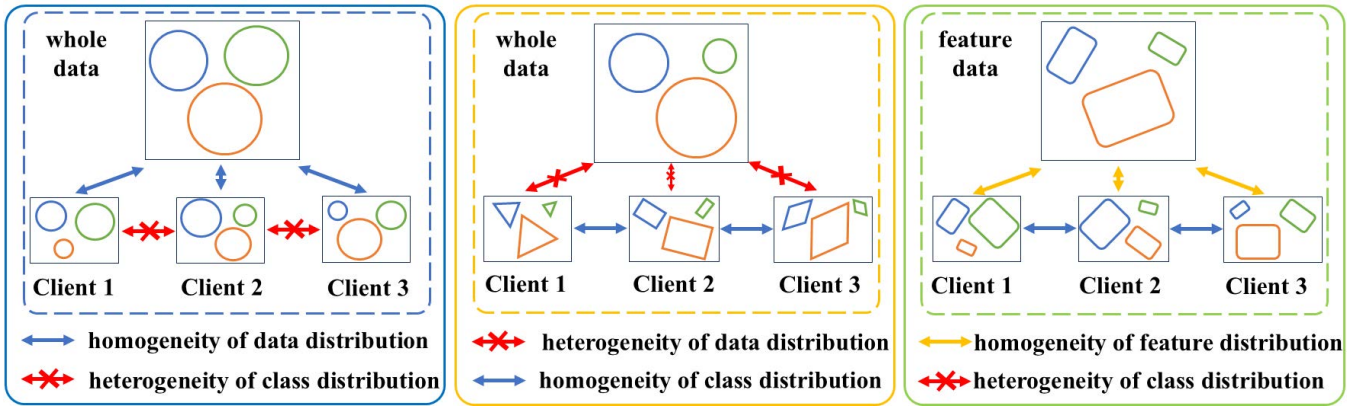
Multi-view data [25, 27, 36, 37] widely exist as heterogeneous information sources that provide complementary and consistent insights from multiple perspectives. For example, medical imaging integrates multiple modalities to characterize pathological features, social networks fuse heterogeneous information to construct user profiles, and industrial monitoring combines sensor data with logs to perceive equipment states and detect anomalies. This cross-modal coupling makes multi-view learning a vital approach to mining deep semantics and overcoming single-view limitations, fueling its broad application in machine learning and data science.

Multi-view clustering [4, 5, 23, 28, 33, 40], as a core unsupervised technique for analyzing complex data structures, aims to collaboratively explore the complementarity and consistency of heterogeneous information to reveal intrinsic clustering patterns. Depending on the degree of information fusion across views, it adopts either a full-view sharing or partial adaptation strategy to

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780182



(a) Horizontal Federated Clustering (b) Vertical Federated Clustering (c) Transposed Vertical Federated Clustering

Figure 1: The specific manifestations of heterogeneity and homogeneity in horizontal and vertical federated clustering (The colors, shapes, and sizes represent different clusters, feature representations, and cluster sizes, respectively.)

construct clustering models. Consequently, multi-view clustering demonstrates significant research value and broad application potential in enhancing clustering performance and handling complex multi-source data.

However, in practical applications, different views of the multi-view data is typically distributed across different data sources and subject to privacy protection restrictions, making traditional centralized multi-view clustering methods [6, 7, 13, 18, 20, 26, 29, 32] difficult to apply directly. Due to privacy regulations and data security requirements, data providers are often unable to share data, which limits the direct access and processing capabilities of traditional methods to global data. Furthermore, there may be significant differences in data storage formats, feature spaces, and distribution characteristics across different views, further increasing the challenges in handling the clustering tasks of this type multi-view data. The technique of handling the distributed multi-view data clustering align with the setting of vertical federated clustering [31].

For a multi-view data $\{D_1, D_2, \ldots, D_l\}$ with l views, the characteristics of Traditional Clustering, Multi-view Clustering, and Vertical Federated Clustering are as follows:

- Traditional Clustering ($\phi_{\mathrm{TC}}(D_i)$): performs clustering on a single view D_i, ignoring information from other views.
- Multi-view Clustering ($\phi_{\mathrm{MC}}(D_1, D_2, \ldots, D_l)$): jointly exploits all views to obtain a unified clustering result.
- Federated Clustering ($\phi_{\mathrm{FC}}(D_i, \varphi(D_1, \ldots, D_l))$): improves local clustering via privacy-preserving collaboration among views, without sharing raw data.

This paper focuses on the Vertical federated clustering problem for multi-view data, where each client holds a single-view dataset with a shared sample space but disjoint feature spaces. The goal is to improve local clustering performance via privacy-preserving collaboration, coordinated by a central server that facilitates cross-view knowledge sharing without exposing raw data.

Vertical federated clustering still faces numerous challenges. First, under the constraints of privacy-preserving mechanisms, there remains a lack of systematic theoretical analysis on whether the data uploaded by clients to the server maintains informational consistency with the original client data. Second, differences in feature spaces across clients make cross-view information sharing and fusion difficult. With regard to federated clustering, we illustrate the heterogeneity and homogeneity characteristics of horizontal and vertical federated clustering in Figure 1. As shown in Figure 1(a), in horizontal federated clustering, client data originates from the same underlying distribution, resulting in homogeneous overall data distribution. However, due to differences in client-side observations, the class distributions across clients exhibit heterogeneity. Specifically, clients share the same feature space, but the proportions of samples in each class vary, leading to inconsistent cluster sizes. This phenomenon is referred to as heterogeneity of class distribution. Figure 1(b) shows that in vertical federated clustering, each client's data represents a subset of features from the entire dataset, leading to heterogeneity in the underlying data distribution—referred to as heterogeneity of data distribution. Nevertheless, as each client observes all data samples, the class distribution remains homogeneous. Addressing these challenges is crucial for building effective and privacy-preserving federated multi-view clustering methods.

To address this, this paper presents an upper bound analysis theory for the k-means loss with respect to the transformation matrix mapping. The theory reveals the relationship between the k-means loss of the data after transformation and the k-means loss of the original data. Furthermore, it is demonstrated that the data transformation method, which performs feature clustering and extracts feature class centers based on k-means, is a lossless mapping method with respect to the k-means loss.

Although the heterogeneity and homogeneity characteristics of horizontal and vertical federated clustering may initially appear unrelated, as illustrated in Figure 1(c), vertical federated clustering exhibits a similar pattern to horizontal federated clustering once the data feature are transposed. For a $n \times d$ data matrix, where n is the data size and d is the feature size. After transforming to a $d \times n$ matrix, treating data features as samples, we find homogeneity of data distribution between clients and the server, and heterogeneity

of class distribution among clients. Comparing Figure 1(c) and 1(a), these homogeneous/heterogeneous relationships match those in Horizontal Federated Clustering.

Inspired by this observation, this paper proposes a vertical federated clustering method that preserves k-means loss. Specifically, each client first transposes its feature matrix to address the challenge of feature space inconsistency. Then, the feature space is expanded accordingly. After this transformation, each client performs k-means clustering on the augmented feature matrix and uploads the obtained cluster centers to the server. The server aggregates the cluster centers, sends the optimized results back to the clients, and collaborates with them for iterative optimization. Finally, the optimized cluster centers are integrated into each client's local data to enhance feature representations and produce the final clustering result. We apply this method to distributed multi-view data, and the effectiveness of the proposed V-HDKM method is validated on 20 multi-view datasets.

The contributions of this paper in terms of methods, theory, and experiments are as follows:

Method: This paper proposes a vertical federated clustering method, which combines feature augmentation of client data with k-means clustering. The cluster centers are aggregated at the server side and undergo collaborative optimization. Ultimately, the optimized cluster centers are integrated into the local data of clients, enhancing data representation and generating the final clustering results.

Theory: This paper introduces a k-means loss upper bound analysis theory for transformation matrix mappings, which reveals what kinds of projection matrices do not compromise the k-means loss.

Experiment: Experimental analysis on 20 multi-view datasets demonstrates the significant improvement of the V-HDKM method on vertical federated clustering performance. Further experimental analysis on 8 UCI benchmark datasets further proves the method's stability.

2 Related Work

In this section, we briefly introduces the related research on Federated Learning, Vertical Federated Learning, and Vertical Federated Clustering.

Federated learning, as a decentralized machine learning paradigm, provides a new solution path for multi-view clustering by establishing a collaborative framework across institutions under privacy protection mechanisms. McMahan et al. proposed the FedAvg algorithm to enable distributed model training in federated learning [34]. Bonawitz et al. introduced a secure multi-party computation-based framework to enhance privacy protection [2]. Li et al. proposed an improved aggregation strategy to accelerate model convergence [24]. Kairouz et al. provided a comprehensive review of the challenges in federated learning, with a focus on data heterogeneity and communication overhead [19]. Federated learning enables the perception and utilization of the global distribution characteristics of data without accessing raw data, thus providing a solid foundation for privacy protection. Based on the distribution characteristics of client data, federated learning can be categorized into horizontal federated learning [16, 45, 46], vertical federated learning [31, 38], and federated transfer learning [39, 41].

In the field of Vertical Federated Learning, numerous methods have been proposed to improve efficiency, privacy preservation, and model performance. For instance, CELU-VFL [11] integrates cached intermediate statistics with a local update mechanism and significantly reduces communication rounds through uniform sampling and instance weighting. VFB^2 [44] introduces a backward update strategy and a bi-level asynchronous parallel architecture, enabling the participation of unlabeled parties in collaborative training, thereby broadening the scope of application. D-Ridge [14] solves ridge regression under the ADMM framework by exchanging only intermediate parameters, effectively safeguarding data privacy. One-shot VFL[35] combines semi-supervised learning with single-round communication and has been extended to Few-shot VFL, which maintains strong performance even with minimal labeled samples. AsySQN [43] applies an asynchronous quasi-Newton method to accelerate convergence. VF^{2}Boost [12] designs a concurrent training protocol with a homomorphic encryption scheme, balancing efficiency and privacy protection. AFSGD-VP [15] propose an asynchronous gradient descent method based on a tree-structured communication topology, mitigating the bottlenecks of traditional synchronous computation. FedAC [42] enhances optimization in heterogeneous data settings via anticipatory momentum and fine-grained gradient correction. FedSDG-FS [21] utilizes Gaussian gating and Gini-based initialization to enable efficient feature selection.

Vertical federated clustering handles scenarios where client data has different feature sets but the same samples, such as different banks sharing different financial information about the same customers. Due to the different data distributions across clients, there are fewer existing methods. DKM [9] constructs a weighted grid based on local clustering results and performs unified clustering on the server in a centralized manner. VKMC [17] generates a weighted coreset through local sensitivity estimation and importance sampling, enabling efficient clustering on the server side. LLFFCM [3] uploads components of local distance matrices, which are aggregated on the server to compute a global distance matrix for clustering. VFCKM [1] merges local centroids and conducts iterative optimization within a shared feature space to maintain clustering consistency. VFDPC [22] employs hybrid encryption to aggregate encrypted distance matrices and merges sub-clusters based on density and similarity, thereby improving clustering performance while preserving privacy.

From the above review, it can be observed that, for vertical federated clustering, the feature space differences among different clients make cross-view information sharing and fusion difficult. In addition, Under privacy protection constraints, the clustering consistency between client-uploaded data and original client data lacks theoretical guarantee.

3 K-Means Cost Bound after projection

Let $D = \{x_1, \ldots, x_n\}$ be the input data with a number of d features and $\mathcal{I} = \{I_1, \ldots, I_K\}$ forms a disjoint partition of the complete set $\{1, \ldots, n\}$, where I_k is the index set of samples in cluster k. That is, the k-th cluster is $D_k = \{x_i \mid i \in I_k\}$ and the complete clustering is

$D = \bigcup_{k=1}^{K} D_k$. Let $\mathrm{ctr}(S)$ denote the center of mass of S. Then the center of cluster is $c_k = \mathrm{ctr}(D_k)$ and the total center is $c = \mathrm{ctr}(X)$. The K-means loss of the partition is,

$$\Delta^2(x; I) = \sum_{k=1}^{K} \sum_{x \in X_k} \|x - c_k\|_2^2 = \sum_{k=1}^{K} \Delta^2(x; I_k). \tag{1}$$

To protect the data, a projection matrix A can be used to map the original data D to transformed space, generating the projected data D' by $D' = DA$. The following properties reveal the relationship between computational loss before and after the transformation.

Theorem 3.1. *Let $\mathcal{I}'_{opt}$ be the optimal partition on the transformed data $D' = DA$, where $D \in \mathbb{A}^{d \times d}$, suppose there exists a γ ($\gamma > 1$) approximation K-means algorithm for D' which can obtain a partition $D'_1 \cup \cdots \cup D'_K$ (the corresponding index partition is $\mathcal{I}' = I'_1 \cup \cdots \cup I'_K$) satisfying:*

$$\Delta^2(x'; I') \leq \gamma \Delta^2(x'; I'_{opt}). \tag{2}$$

then the k-means loss of obtained partition on the original data satisfies:

$$\Delta^2(x; I') \leq \gamma \kappa(AA^T) \Delta^2(x; I_{opt}). \tag{3}$$

where $\kappa(AA^T) = \frac{\sigma_{max}(AA^T)}{\sigma_{min}(AA^T)}$ is the condition number of AA^T, and σ is singular value.

The proof of Theorem 3.1 is in Appendix A.1. From Theorem 3.1, we deduce that if $\mathcal{I}'$ is a γ-approximate partition in the transformed space, then $\mathcal{I}'$ is a $\gamma\kappa(AA^T)$-approximate partition in the original space. For well-conditioned transformations ($\kappa \approx 1$), such as orthogonal transformations, the K-means loss of $\mathcal{I}'$ on D remains small.

Guided by this property, we can design projection methods on the clients in federated clustering that have minimal impact on the k-means loss, thereby preserving data privacy. We then develop a Vertical Federated K-Means method in the following section.

4 Vertical Federated K-means

In this subsection, we will elaborate on the implementation process of the Vertical Federated K-means (V-HDKM) method. The V-HDKM method is designed to handle data distributed across multiple clients, where clients share samples but exclusively own different feature dimensions in a vertical federated clustering scenario. This method optimizes clustering performance by leveraging feature transposition and incorporating distribution-aware techniques from horizontal federated clustering. The overall framework of V-HDKM is illustrated in Figure 2.

The data from all clients can be collectively represented as $\{n * d_1, n * d_2, \cdots, n * d_l\}$, where n denotes the number of data points per client, d_i represents the feature dimension of client C_i, and l is the total number of clients. Specifically, client C_i holds a dataset $D_i = \{x_1, x_2, \ldots, x_n\} \in \mathbb{R}^{n \times d_i}$. Sample x_i can be represented as $\{f_1, f_2, \ldots, f_d\}$, where f_j denotes a d-dimensional space feature.

4.1 Client Feature Preprocessing

First, the feature matrix of each client, D_i, is transposed to obtain $D_i^T \in \mathbb{R}^{d_i \times n}$, providing support for subsequent feature expansion and integration operations:

$$D_i \in \mathbb{R}^{n \times d_i} \Rightarrow D_i^T \in R^{d_i \times n}. \tag{4}$$

Then, guided by Theorem 3.1, we expanded the client data features. To enhance the diversity of the data, d_i times random sampling with replacement is performed on the rows of the matrix D_i^T, and the mean of the sampled rows is calculated to generate a new feature vector in the ith client:

$$\hat{x}'_{ij} = \frac{1}{d_i} \sum_{j=1}^{d_i} D_i^T[s_j], \tag{5}$$

where s_j denotes the row index obtained from the j-th random sampling, and $D_i^T[s_j]$ represents the s_j-th row of the matrix .

The computation process for the new feature vector is repeated k^2 times, resulting in the final expanded feature matrix $\hat{X}_i \in \mathbb{R}^{k^2 \times n}$. The expanded feature matrix $\hat{X}_i$ is then concatenated with the transposed feature matrix D_i^T to obtain the final integrated matrix:

$$D_i^{new} = \begin{bmatrix} D_i^T \\ \hat{X}_i \end{bmatrix}, \quad D_i^{new} \in \mathbb{R}^{(d_i + k^2) \times n}. \tag{6}$$

The integrated matrix D_i^{new} combines the original features and the expanded features, allowing for a more comprehensive representation of the characteristics of the local data. Each sample in D_i^{new} can be represented as $\{f_1, \ldots, f_{d_i}, f_{d_i+1}, \ldots, f_{d_i+k^2}\}$. The discussion of the rationality of D_i^{new} is given in Appendix A.2. The training is then performed under the H-HDKM module.

4.2 H-HDKM: Client-Side

Local clustering: The client performs K-means clustering on its data D_i^{new}, calculating the Euclidean distance from each data point to all cluster centers and assigning each point to the nearest cluster center, until the change in cluster centers is below a threshold or the maximum number of iterations is reached. The new cluster centers are determined by calculating the mean of all data points in each cluster, as follows:

$$n_{ij} = \sum_{d=1}^{d_i+k^2} \mathbb{I}\{x_d \in S_j\}, \tag{7}$$

$$u_{ij} = \frac{1}{n_{ij}} \sum_{d=1}^{d_i+k^2} \{x_d | x_d \in S_j\}. \tag{8}$$

Here, n_{ij} represents the number of data points in the j-th cluster of the i-th client, u_{ij} denotes the cluster center of the j-th cluster, and S_j is the set of data points assigned to the j-th cluster.

When local K-means clustering is complete, each client generates preliminary clustering information, including the cluster centers $u_i = \{u_{i1}, u_{i2}, \ldots, u_{ik}\}$ and the corresponding sample counts $n_i = \{n_{i1}, n_{i2}, \ldots, n_{ik}\}$, which are then uploaded to the server. The analysis of the rationality of the uploaded information u_i is given in Appendix A.2.

Global alignment: Each client C_i receives the global cluster centers $\{\mu_1^t, \mu_2^t, \ldots, \mu_k^t\}$ from the server. The client then reassigns its local data points based on the global information. Specifically, for each local data point x_i, the Euclidean distance to all global cluster centers is calculated as follows:

$$d_{ij} = \left\| x_i - \mu_j \right\|^2, \tag{9}$$

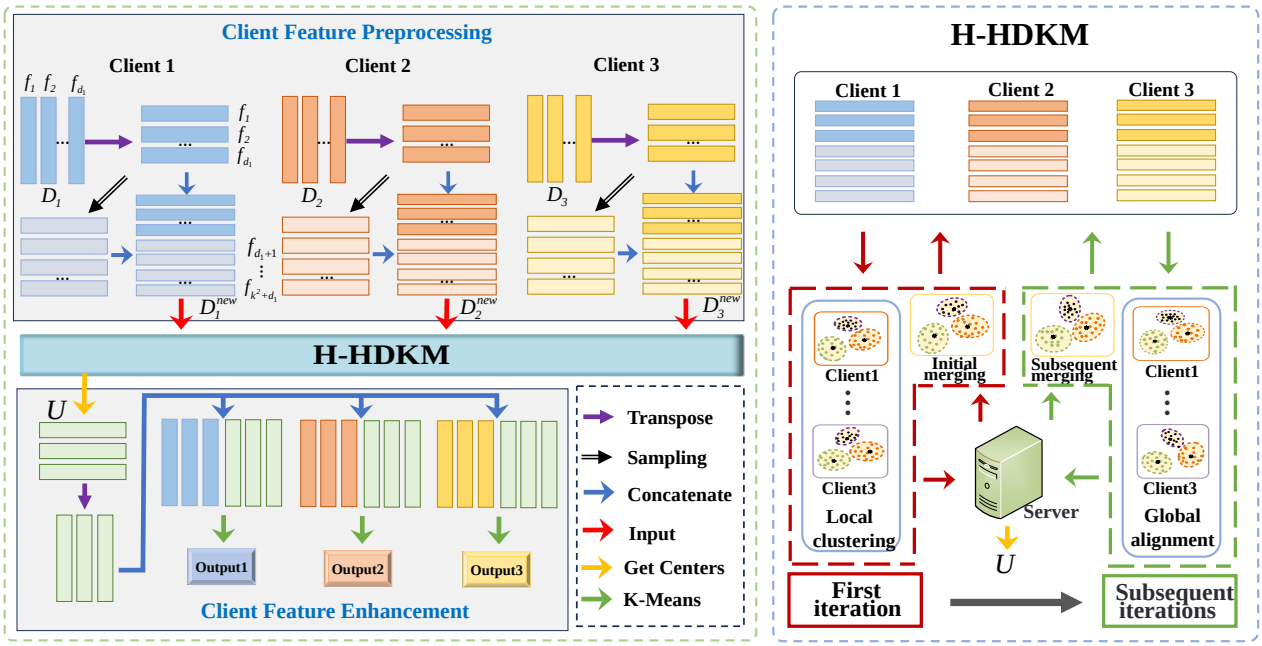


Figure 2: The proposed V-HDKM method is summarized as follows: the overall framework of V-HDKM is shown on the left. In this approach, each client first performs feature transposition and expansion on its local data D_i to obtain a new data representation D_i^{new}. Subsequently, the H-HDKM module is employed to compute the cluster centers U. The client then integrates the optimized cluster centers into its local data to enhance the data representation and generate the final clustering results. The detailed process of H-HDKM is illustrated on the right.

here, d_{ij} represents the distance from the data point x_i to the global cluster center μ_j. Each data point is then assigned to the nearest cluster center, and the clustering information is updated, including the sample count, cluster center location, and radius:

$$n_{ij}^{(t+1)} = \sum_{d=1}^{d_i+k^2} \mathbb{I}\{x_d \in S_j\}, \tag{10}$$

$$u_{ij}^{(t+1)} = \frac{1}{n_{ij}^{(t+1)}} \sum_{d=1}^{d_i+k^2} \{x_d | x_d \in S_j\}, \tag{11}$$

$$r_{ij}^{(t+1)} = \frac{1}{n_{ij}^{(t+1)}} \sum_{d=1}^{d_i+k^2} \{||\, x_d - \mu_j \,|| \;\; | x_d \in S_j\}. \tag{12}$$

Here, $r_{ij}^{(t+1)}$ represents the radius of the j-th cluster of client C_i, indicating the average distance from the data points to the cluster center. If a cluster has no data points assigned, the relevant information will be deleted.

After the local data reallocation is completed, the client uploads the updated clustering information to the server, including the new cluster centers $u_i = \{u_{i1}, u_{i2}, \ldots, u_{ik}\}$, sample counts $n_i = \{n_{i1}, n_{i2}, \ldots, n_{ik}\}$, and cluster radii $r_i = \{r_{i1}, r_{i2}, \ldots, r_{ik}\}$.

4.3 H-HDKM: Server-Side

The server receives the uploaded cluster centers u_i, sample counts n_i, and radii r_i from each client, and divides the aggregation process into two phases: initial merging and subsequent merging. The initial merging refers to the first merge, while subsequent merging begins from the second round. In either phase, the first step for the server is to merge the cluster centers from all clients, forming a global cluster center set: $\{\mu_{11}^t, \mu_{12}^t, \cdots, \mu_{lk}^t\}$, where each $\mu_{ij}^t \in \mathbb{R}^n$.

Initial merging: In this process, the cluster centers uploaded by each client are treated as data points in the global dataset, and the server applies the weighted K-Means algorithm to further cluster these cluster centers. The weighted K-Means defines the weights of the sample points by considering the sample count of each cluster center uploaded by the client. By using weighted clustering, the distribution differences of the cluster centers uploaded by the clients are fully accounted for, resulting in the final merged k global cluster centers: $u = \{\mu_1, \mu_2, \ldots, \mu_k\}$. The sample count for each cluster center μ_m is the total number of samples from all client cluster centers belonging to that cluster, calculated as follows:

$$n_i = \sum_j \mathbb{I}\{\mu_{ij} \in \mu_i\}. \tag{13}$$

After completing the initial merging phase, the server distributes the generated global cluster center set $u = \{\mu_1, \mu_2, \ldots, \mu_k\}$ and the corresponding sample count set $n = \{n_1, n_2, \ldots, n_k\}$ to each client for the next round of iterative training.

Subsequent merging: To ensure that each cluster center μ_{ij}^t uploaded by the client is correctly assigned to the most matching global cluster center, the server calculates the Euclidean distance between each cluster center uploaded by the client and the global

cluster centers from the previous round, $\{\mu_1^{t-1}, \mu_2^{t-1}, \ldots, \mu_k^{t-1}\}$:

$$d = \left\|\mu_{ij} - \mu_m^{t-1}\right\|^2. \tag{14}$$

Based on the calculation results, each cluster center μ_{ij} uploaded by the client is assigned to the nearest global cluster center μ_m^{t-1}.

$$S_m = \arg\min_m \left\|\mu_{ij} - \mu_m^{t-1}\right\|^2. \tag{15}$$

After the assignment is completed, the server updates the global cluster centers, sample counts, and radii as follows:

$$\mu_m^t = \frac{\sum \mu_{ij} n_{ij}}{\sum n_{ij}}, \tag{16}$$

$$n_m^t = \sum_i \mathbb{I}\{\mu_{ij} \in \mu_i^{t-1}\}, \tag{17}$$

$$r_m^t = \frac{\sum r_{ij} n_{ij}}{\sum n_{ij}}. \tag{18}$$

Through these steps, the server completes the optimization of the global cluster centers and sends the updated information to each client, serving as the basis for the next round of clustering.

4.4 H-HDKM: Iterative phase

In the iterative phase, the algorithm optimizes the clustering results through multiple rounds of communication between the clients and the server, ensuring data privacy protection while improving the accuracy and consistency of the clustering. The iteration continues until the change in the cluster centers is negligible, meeting the convergence threshold. Finally, the cluster centers stabilize, yielding the optimal clustering result for the global data: $U = \{\mu_1, \mu_2, \ldots, \mu_k\}$.

Then, the global cluster center matrix $U = \{\mu_1, \mu_2, \ldots, \mu_k\}$, $U \in \mathbb{R}^{k \times n}$ is transposed to obtain the transposed matrix U^T:

$$U^T = [\mu_1^T, \mu_2^T, \ldots \mu_k^T]^T, \quad U^T \in \mathbb{R}^{n \times k}. \tag{19}$$

4.5 Client Feature Enhancement

The transposed global cluster center matrix $U^T \in \mathbb{R}^{n \times k}$ is distributed to each client and concatenated with the client's original data matrix D_i to form the final client data matrix:

$$D_i^{final} = \begin{bmatrix} D_i & U^T \end{bmatrix}, \quad D_i^{final} \in \mathbb{R}^{n \times (d_i + k)}. \tag{20}$$

The matrix X_i^{final} integrates both the client's original features and the global cluster center information, providing a more comprehensive representation of the sample features. Based on this, each client performs K-Means clustering again on the integrated data to compute the final clustering metrics, thereby evaluating the effectiveness of the global clustering results.

The process of V-HDKM is presented in Algorithm 1 .

Algorithm 1 V-HDKM

Input: Client feature matrices $\{D_i\}_{i=1}^l$, Cluster K
Output: The concatenated data matrix at the client side D_i^{final}
Process:
Client processing:
for each client $i = 1$ to l **do**
  Transpose $D_i \in \mathbb{R}^{n \times d_i}$ to $D_i^T \in R^{d_i \times n}$
  for $j = 1$ to k^2 **do**
    Compute $\hat{x}_{ij}'$ based on Equation(5)
  end for
  Get $D_i^{new} \leftarrow \begin{bmatrix} D_i^T \\ \hat{X}_i \end{bmatrix}$
end for
Execute H-HDKM Module:
// Client-side operations
if Local clustering **then**
  for each client i in parallel **do**
    $(u_i, n_i) \leftarrow$ k-means(D_i^{new}, k)
    Send (u_i, n_i) to server
  end for
else
  for each client i in parallel **do**
    $n_i \leftarrow$ Calculate based on Equation(10)
    $u_i \leftarrow$ Calculate based on Equation(11)
    $r_i \leftarrow$ Calculate based on Equation(12)
    Send (n_i, u_i, r_i) to server
  end for
end if
// Server-side operations
if Initial merging **then**
  Aggregate $\{u_i\}, \{n_i\}$ from all clients
  $(u, n) \leftarrow$ Weighted K-Means$(u, \text{weights} = n_i, k)$
else
  $u_m^t \leftarrow$ Update based on Equation(16)
  $n_m^t \leftarrow$ Update based on Equation(17)
  $r_m^t \leftarrow$ Update based on Equation(18)
Broadcast updated (u, n, r) to all clients
end if
Iterative phase: Get global centers $U = \mu_1, \mu_2, \ldots, \mu_k$
Postprocessing:
$U^T \leftarrow$ Transpose global centers U
for each client $i = 1$ to l **do**
  $D_i^{final} \leftarrow \begin{bmatrix} D_i & U^T \end{bmatrix}$
end for

5 Experiments

In this section, the V-HDKM method is systematically validated. First, its performance is evaluated on 20 multi-view datasets. The robustness and parameter adaptability of the proposed method are evaluated. Additional experiments are conducted on 8 UCI benchmark datasets to evaluate the stability of the method.

5.1 Experimental Settings

5.1.1 Experimental Dataset. To evaluate the performance of the proposed method, experiments were conducted on multiple publicly available multi-view and single-view datasets. Table 1 lists 20 multi-view datasets, labeled from 1 to 20, including basic information such as the number of samples (n), the number of classes (k), and the number of views (v), offering good diversity and representativeness.

5.1.2 Compared Methods. To evaluate the clustering performance of V-HDKM, seven representative vertical federated clustering

Table 1: Description of the Multi-view datasets

Number	Data name	n	k	v
1	100Leaves	1600	100	3
2	10X_PBMC	4271	8	9
3	ALOI_100	10800	100	4
4	best_orl	400	40	3
5	Caltech101-20	2386	20	6
6	Caltech101-7	1474	7	6
7	COIL20-3v	1440	20	3
8	FACS_v2_Heart	3421	11	9
9	FACS_v2_Large_Intestine-counts	3362	15	9
10	handwritten	2000	10	6
11	HumanKidney	5685	11	9
12	MNIST_10000n_Xs	10000	10	3
13	MSRC_v1	210	7	5
14	ORL	400	40	4
15	scene	2688	8	4
16	scene-15	4485	15	3
17	stl10_fea	13000	10	3
18	uci_digit	2000	10	3
19	WebKB	203	4	3
20	Wiki_fea	2865	10	2

Table 2: Description of the UCI data sets

Number	Data name	n	d	k
D1	breast-cancer-wisc-diag	569	30	2
D2	breast-cancer-wisc	699	9	2
D3	led-display	1000	7	10
D4	optical_test	1797	62	10
D5	statlog-image	2310	18	7
D6	pendigits_test	3498	16	10
D7	optical_train	3823	62	10
D8	pendigits_train	7494	16	10

methods were selected as comparative baselines, including: K-Means Clustering with Distributed Dimensions (DKM [9]), Coresets for Vertical Federated Learning: Regularized Linear Regression and K-Means Clustering (VKMC [17]), Federated c-Means and Fuzzy c-Means Clustering Algorithms for Horizontally and Vertically Partitioned Data (LLFFCM [3]), a federated clustering framework based on k means algorithm for vertically partitioned data with shared attributes (VFCKM [1]), Vertical Federated Density Peaks Clustering Under Nonlinear Mapping (VFDPC [22]), Efficient Federated Multi-View Clustering with Integrated Matrix Factorization and K-Means (FMVC-IMK [10]), Efficient Federated Incomplete Multi-View Clustering (EFIMVC [30]).

In the experiments, each client C_i corresponds to a specific view and holds the data from that view. Under this multi-view partitioning setting, the number of clients equals the number of views, and the number of clusters corresponds to the number of classes in the multi-view dataset.

5.1.3 Evaluation Metrics. To assess the clustering performance of the method, we used three common evaluation metrics: Adjusted Rand Index (ARI), Normalized Mutual Information (NMI), and Accuracy (ACC). The ARI, NMI, and ACC values listed in the table represent the average clustering performance across the four clients, with each client's result averaged over 30 independent runs to reduce the impact of randomness and ensure the reliability of the experimental outcomes.

5.2 Experimental Results and Analysis

5.2.1 Comparison Results. Firstly, we conduct experiments on multiple multi-view datasets to evaluate the performance of the proposed V-HDKM method and that of local clients under different clustering algorithms. As shown in Table 3, we perform clustering analysis on multi-view data using K-Means as the clustering algorithm. Here, "Local" denotes the average performance obtained by independently running K-Means on the local data of four individual clients, while "V-HDKM" represents our proposed method.

As shown in Table 3, V-HDKM significantly outperforms the Local method across all 20 datasets. Taking the NMI metric as an example, V-HDKM achieves an average score of 0.562, representing a 7% improvement over the Local method's 0.492. For ACC (clustering accuracy) and ARI (Adjusted Rand Index), the average scores of V-HDKM are 0.536 and 0.399, respectively, which are 6.2% and 8.5% higher than those of the Local method. Compared to Local, V-HDKM consistently delivers superior clustering performance across the multi-view datasets.

In addition, we respectively adopt Hierarchical Clustering and Spectral Clustering as the base algorithms. Tables 4 and Tables 5 respectively present the clustering results of the V-HDKM and Local methods on multi-view datasets under the same experimental settings. The experimental results indicate that, regardless of the clustering method used, V-HDKM outperforms the Local method in most cases, further validating its effectiveness under different clustering algorithms.

These results demonstrate that V-HDKM can effectively integrate information from different views, thereby enhancing clustering consistency and overall performance.

5.2.2 Violin Charts. To further analyze and demonstrate the performance of the proposed method, we plotted violin charts showing the mean and variance over 30 experimental runs. Figure 3 illustrates the clustering performance of V-HDKM and the Local method on multi-view datasets 1-5, evaluated using the NMI metric as summarized in Table 3. As shown in these figures, V-HDKM consistently achieves higher mean values, indicating better overall performance. On most datasets, the distribution ranges of V-HDKM and the Local method are similar, suggesting comparable stability—likely due to both methods being based on the k-means algorithm. In contrast, V-HDKM shows greater variance on these datasets, indicating a higher potential for performance enhancement.

5.2.3 The impact of the number of clients. Since the number of clients in multi-view datasets is fixed (corresponding to the number of views), we use UCI benchmark datasets to simulate vertical

Table 3: The Indices Values from the Compared Methods on the Twenty Datasets

Method	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	Avg
NMI																					
DKM	0.748	0.505	0.401	0.782	0.358	0.361	0.694	**0.603**	0.390	0.532	0.357	0.221	0.396	0.779	0.133	0.256	0.001	0.629	0.173	0.352	0.417
VKMC	0.691	0.438	0.570	0.668	0.393	0.354	0.668	0.473	0.409	0.516	0.351	0.343	0.421	0.648	0.254	0.249	0.285	0.547	0.143	0.251	0.420
LLF-FCM	0.669	0.322	0.399	0.667	0.337	0.354	0.594	0.349	0.364	0.509	0.257	0.163	0.361	0.657	0.145	0.206	0.104	0.471	0.144	0.375	0.357
VFCKM	0.739	0.501	0.670	0.783	0.459	0.397	0.786	0.485	0.431	0.602	0.372	0.414	0.434	0.751	0.341	0.307	0.308	0.613	0.190	0.256	0.492
VFDPC	0.404	0.378	0.292	0.473	0.353	0.417	0.396	0.312	0.307	0.300	0.216	0.327	0.458	0.445	0.369	0.339	0.264	0.404	**0.428**	0.350	0.350
FMVC-IMK	0.294	0.148	0.272	0.243	0.064	0.056	0.368	0.094	0.072	0.138	0.085	0.371	0.045	0.614	0.087	0.216	0.133	0.324	0.348	0.247	0.211
EFIMVC	0.725	0.465	0.618	0.620	0.440	0.443	0.640	0.483	0.425	0.593	0.301	0.083	0.556	0.673	0.464	0.264	0.316	0.558	0.308	0.036	0.451
Local	0.739	0.501	0.670	0.783	0.459	0.397	0.786	0.485	0.431	0.602	0.372	0.414	0.434	0.751	0.341	0.307	0.308	0.613	0.190	0.256	0.492
V-HDKM	**0.849**	**0.536**	**0.737**	**0.790**	**0.573**	**0.531**	**0.792**	0.511	**0.447**	**0.688**	**0.380**	**0.433**	**0.565**	**0.785**	**0.481**	**0.379**	**0.367**	**0.680**	0.308	**0.411**	**0.562**
ACC																					
DKM	0.489	0.489	0.151	0.607	0.282	0.453	0.516	**0.541**	0.320	0.360	0.423	0.336	0.438	0.610	0.281	0.262	0.111	0.642	0.591	0.421	0.416
VKMC	0.397	0.473	0.318	0.437	0.387	0.473	0.501	0.464	0.358	0.419	0.434	0.431	0.490	0.431	0.372	0.257	0.350	0.555	0.610	0.324	0.424
LLF-FCM	0.358	0.395	0.162	0.431	0.313	0.464	0.427	0.358	0.356	0.384	0.401	0.268	0.430	0.416	0.300	0.229	0.251	0.451	0.560	0.400	0.368
VFCKM	0.477	0.501	0.429	0.605	0.356	0.420	0.644	0.420	0.366	0.527	0.438	0.511	0.511	0.564	0.434	0.294	0.378	0.642	0.621	0.331	0.474
VFDPC	0.117	0.369	0.083	0.225	0.263	0.443	0.272	0.308	0.259	0.268	0.364	0.338	0.495	0.205	0.396	0.275	0.294	0.379	0.447	**0.512**	0.316
FMVC-IMK	0.423	0.225	0.374	0.398	0.202	0.098	0.588	0.193	0.169	0.242	0.125	0.476	0.126	0.355	0.145	0.260	0.372	0.443	0.586	0.352	0.306
EFIMVC	0.454	0.460	0.411	0.383	0.347	0.413	0.519	0.427	0.360	0.476	0.415	0.217	0.595	0.440	0.556	0.314	0.400	0.596	0.605	0.167	0.428
Local	0.477	0.507	0.429	0.605	0.356	0.420	0.644	0.420	0.366	0.527	0.438	0.511	0.511	0.564	0.434	0.294	0.378	0.642	0.621	0.331	0.474
V-HDKM	**0.688**	**0.504**	**0.549**	**0.619**	**0.415**	**0.476**	**0.661**	0.432	**0.369**	**0.590**	**0.453**	**0.545**	**0.642**	**0.613**	**0.569**	**0.357**	**0.426**	**0.725**	**0.629**	0.449	**0.536**
ARI																					
DKM	0.361	0.361	0.080	0.453	0.165	0.255	0.406	**0.426**	0.164	0.393	0.296	0.127	0.259	0.468	0.084	0.109	0.001	0.506	0.092	0.179	0.259
VKMC	0.255	0.264	0.177	0.227	0.257	0.287	0.389	0.270	0.172	0.365	0.226	0.239	0.281	0.258	0.172	0.106	0.259	0.399	0.152	0.177	0.247
LLF-FCM	0.223	0.187	0.088	0.255	0.197	0.291	0.318	0.165	0.191	0.313	0.176	0.080	0.205	0.236	0.080	0.078	0.057	0.306	0.081	0.230	0.188
VFCKM	0.341	0.333	0.258	0.452	0.250	0.282	0.583	0.252	0.195	0.485	0.265	0.326	0.298	0.422	0.233	0.149	0.297	0.498	0.186	0.180	0.314
VFDPC	0.011	0.214	0.030	0.054	0.137	0.223	0.147	0.151	0.130	0.145	0.262	0.174	0.287	0.037	0.211	0.139	0.133	0.221	0.110	0.128	0.147
FMVC-IMK	0.423	0.374	0.273	0.243	0.194	0.290	0.478	0.276	0.223	0.304	0.266	0.244	0.276	0.206	0.298	0.099	0.096	0.501	0.319	0.165	0.277
EFIMVC	0.310	0.331	0.276	0.210	0.294	0.282	0.433	0.297	0.228	0.450	0.265	0.194	0.436	0.277	0.339	0.150	0.175	0.402	0.335	0.202	0.294
Local	0.341	0.333	0.258	0.452	0.250	0.282	0.583	0.252	0.195	0.485	0.265	0.326	0.298	0.422	0.233	0.149	0.297	0.498	0.186	0.180	0.314
V-HDKM	**0.578**	**0.393**	**0.415**	**0.474**	**0.341**	**0.384**	**0.605**	0.312	**0.234**	**0.570**	**0.307**	**0.337**	**0.448**	**0.477**	**0.356**	**0.201**	**0.336**	**0.579**	**0.355**	**0.273**	**0.399**

Table 4: The Indices Values from the Hierarchical-based V-HDKM on the Twenty Datasets

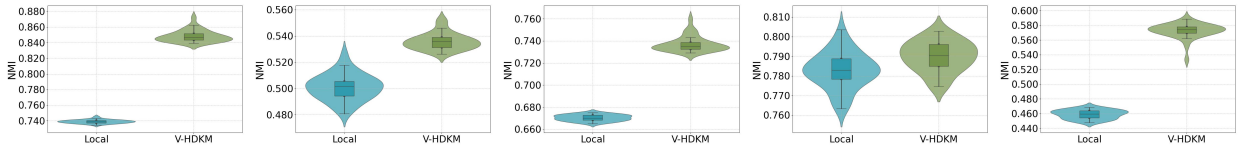
Metric	Method	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	Avg
NMI	Local	0.765	0.547	0.739	0.737	0.485	0.415	0.555	0.521	0.407	0.649	0.331	0.451	0.466	0.813	0.337	0.306	0.311	0.685	0.243	0.256	0.501
	V-HDKM	**0.883**	**0.553**	**0.778**	**0.781**	**0.605**	**0.572**	**0.678**	**0.538**	**0.456**	**0.716**	**0.389**	**0.500**	**0.631**	**0.843**	**0.489**	**0.377**	**0.412**	**0.727**	**0.295**	**0.412**	**0.582**
ACC	Local	0.512	**0.538**	0.581	0.480	0.390	0.451	0.301	0.439	0.342	0.535	0.442	0.538	0.529	0.641	0.434	0.301	0.398	0.685	0.615	0.344	0.475
	V-HDKM	**0.738**	0.525	**0.632**	**0.535**	**0.446**	**0.536**	**0.373**	**0.447**	**0.369**	**0.599**	**0.476**	**0.587**	**0.694**	**0.685**	**0.550**	**0.354**	**0.476**	**0.766**	**0.645**	**0.452**	**0.544**
ARI	Local	0.382	0.380	0.317	0.571	0.282	0.284	0.258	0.286	0.173	0.515	0.243	0.344	0.310	0.535	0.234	0.146	0.312	0.567	0.256	0.185	0.329
	V-HDKM	**0.644**	**0.404**	**0.320**	**0.577**	**0.387**	**0.430**	**0.340**	**0.335**	**0.253**	**0.582**	**0.310**	**0.392**	**0.517**	**0.574**	**0.352**	**0.200**	**0.385**	**0.613**	**0.334**	**0.278**	**0.411**

Table 5: The Indices Values from the Spectral-based V-HDKM on the Twenty Datasets

Metric	Method	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	Avg
NMI	Local	0.764	0.494	0.642	0.826	0.464	0.420	0.819	0.458	0.411	0.724	0.333	0.501	0.484	0.793	0.385	0.333	0.316	0.762	0.235	0.258	0.521
	V-HDKM	**0.884**	**0.529**	**0.799**	**0.836**	**0.593**	**0.537**	**0.861**	**0.498**	**0.436**	**0.851**	**0.372**	**0.571**	**0.632**	**0.801**	**0.538**	**0.394**	**0.417**	**0.842**	**0.296**	**0.428**	**0.606**
ACC	Local	0.561	**0.500**	0.333	0.686	0.381	0.519	0.587	0.380	**0.360**	0.602	0.425	0.530	0.569	0.635	0.449	0.307	0.401	0.746	0.527	0.317	0.491
	V-HDKM	**0.787**	0.499	**0.612**	**0.710**	**0.443**	**0.543**	**0.651**	**0.417**	**0.360**	**0.741**	**0.447**	**0.599**	**0.720**	**0.654**	**0.589**	**0.359**	**0.445**	**0.899**	**0.617**	**0.447**	**0.577**
ARI	Local	0.408	0.309	0.226	0.538	0.227	0.317	0.524	0.206	0.166	0.600	0.221	0.324	0.350	0.492	0.250	0.157	0.318	0.629	0.254	0.128	0.332
	V-HDKM	**0.677**	**0.384**	**0.520**	**0.571**	**0.322**	**0.387**	**0.590**	**0.274**	**0.217**	**0.788**	**0.288**	**0.416**	**0.537**	**0.519**	**0.376**	**0.186**	**0.361**	**0.806**	**0.277**	**0.257**	**0.438**

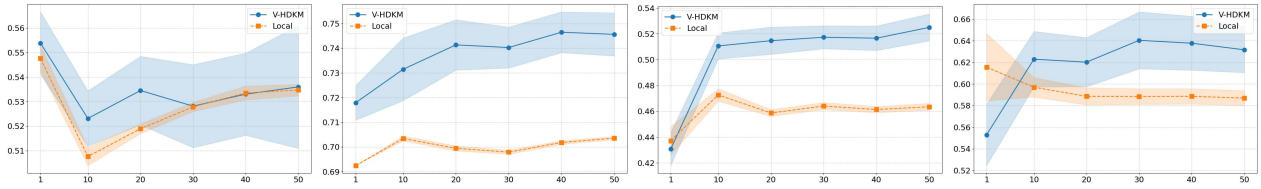
federated clustering scenarios in order to analyze the impact of the number of clients on the proposed method. Table 2 presents 8 UCI benchmark datasets, labeled from D1 to D8, providing information on the number of samples (n), feature dimensions (d), and the number of classes (k).

Based on these datasets, we set the number of clients to 1, 10, 20, 30, 40, and 50. Each client C_i holds only a subset of the dataset's features, meaning its local data is constructed by sampling features with replacement.



(a) Data 1-NMI (b) Data 2-NMI (c) Data 3-NMI (d) Data 4-NMI (e) Data 5-NMI

Figure 3: Violin Plots of NMI Scores Across Multi-view Data 1-5



(a) Data D1-NMI (b) Data D2-NMI (c) Data D3-NMI (d) Data D4-NMI

Figure 4: Parameter Sensitivity Analysis in Term of NMI for D1-D4

Figure 4 shows the impact of the number of clients analysis of the V-HDKM method compared to the Local method on the UCI dataset D1-D4, with the evaluation metric being NMI. As shown in these figures, as the number of clients increases from 1 to 50, V-HDKM consistently achieves higher scores than the Local method, with a significantly narrower performance fluctuation range. Especially when the number of clients is large, V-HDKM maintains high clustering quality, whereas the performance of the Local method declines. In contrast, the performance of the Local method is more sensitive to the number of clients, which indicates that V-HDKM is more adaptable to changes in the number of clients and can maintain robust performance across distributed environments of varying scales.

Overall, the experimental results show that V-HDKM outperforms the Local method on most datasets. Moreover, its performance exhibits less fluctuation as the number of clients changes, demonstrating superior stability.

5.2.4 Complexity. Time Complexity: The overall process of H-HDKM is predominantly grounded in the principles of the k-means algorithm, which contributes to its high efficiency in terms of time complexity. The time complexity of client side initialization is $O(l \times k^3 \times t_1 \times n)$, where l is the number of clients, k is the number of clusters, n is the number of data samples, t_1 is the number of iterations. The time complexity of the interaction between clients and the server is $O(t_2 \times n \times l \times (kt_3 + k^2)$, where t_2 is the number of iterations in this strategy. If the k-means algorithm is utilized in the client clustering step, the time cost is $O(n \times k \times t_3)$, where t_3 is the number of iterations. The total time complexity is the sum of the above three parts.

Communication Complexity: Each client uploads its cluster centers, leading to a communication complexity of $O(tlkn)$, where l, k, n, and t denote the numbers of clients, clusters, samples, and iterations, respectively. The server then returns fused global centers with the same complexity.

5.2.5 Privacy Analysis. In H-HDKM, clients interact with the server by exchanging feature center information. Uploading cluster enters to the server is a well established privacy preserving strategy in the field of federated clustering, and this approach has been widely adopted in numerous existing methods [8]. Beyond that, our strategy further strengthens privacy protection by enriching the feature space through the averaging of multiple random samplings. This additional measure effectively obscures the original data characteristics.

6 Conclusion

This paper addresses the distributed storage and privacy protection requirements of multi-view data in real-world applications, which aligns with the scenario setup of vertical federated learning. To address the challenge of theoretically analyzing the clustering consistency between the data uploaded by clients to the server and the original client data, we first establish a k-means loss bound for the projected data. This bound reveals the relationship between the k-means loss on the transformed data and that on the original data. Guided by this theoretical insight, we propose a vertical federated clustering method named V-HDKM. This method first mitigates the information fusion challenge caused by discrepancies in feature spaces across views by transposing the feature matrices. Then, each client performs feature transformation guided by the theory—first expanding and then compressing the features. Finally, the method achieves cross-view clustering consistency through the collaborative optimization of clustering centers. We comprehensively evaluate V-HDKM on 20 multi-view datasets. Experimental results confirm its effectiveness in achieving high-quality clustering comparable to centralized methods and other vertical federated methods. Furthermore, tests on UCI datasets demonstrate the stability and generalizability of our method.

Future work will focus on designing more communication-efficient mechanisms for cluster center transfer and exploring broader strategies for modeling and integrating data from highly heterogeneous views.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (62441239, 62476160, 62306170, U24A20253, T2495251, 62136005), the Science and Technology Major Project of Shanxi (202201020101006), the Special Fund for Science and Technology Innovation Teams of Shanxi Province (202304051001001).

References

[1] Oruba Alfawaz, Ali A El-Moursy, Mohamed Saad, and Ahmed M Khedr. 2025. VFCkM: a federated clustering framework based on k-means algorithm for vertically partitioned data with shared attributes. *The Journal of Supercomputing* 81, 7 (2025), 1–25.

[2] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konečnỳ, Stefano Mazzocchi, Brendan McMahan, et al. 2019. Towards federated learning at scale: System design. *machine learning and systems* 1 (2019), 374–388.

[3] José Luis Corcuera Bárcena, Francesco Marcelloni, Alessandro Renda, Alessio Bechini, and Pietro Ducange. 2024. Federated *c*-Means and Fuzzy *c*-Means Clustering Algorithms for Horizontally and Vertically Partitioned Data. *IEEE Transactions on Artificial Intelligence* 5, 12 (2024), 6426–6441. doi:10.1109/TAI.2024.3426408

[4] Guoqing Chao, Yi Jiang, and Dianhui Chu. 2024. Incomplete contrastive multi-view clustering with high-confidence guiding. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 11221–11229.

[5] Kamalika Chaudhuri, Sham M. Kakade, Karen Livescu, and Karthik Sridharan. 2009. Multi-view clustering via canonical correlation analysis. In *Proceedings of the 26th Annual International Conference on Machine Learning* (Montreal, Quebec, Canada) *(ICML '09)*. Association for Computing Machinery, New York, NY, USA, 129–136. doi:10.1145/1553374.1553391

[6] Miaomiao Cheng, Liping Jing, and Michael K. Ng. 2019. Tensor-Based Low-Dimensional Representation Learning for Multi-View Clustering. *IEEE Transactions on Image Processing* 28, 5 (2019), 2399–2414. doi:10.1109/TIP.2018.2877937

[7] Chenhang Cui, Yazhou Ren, Jingyu Pu, Jiawei Li, Xiaorong Pu, Tianyi Wu, Yutao Shi, and Lifang He. 2023. A novel approach for effective multi-view clustering with information-theoretic perspective. *Advances in neural information processing systems* 36 (2023), 44847–44859.

[8] Don Kurian Dennis, Tian Li, and Virginia Smith. 2021. Heterogeneity for the Win: One-Shot Federated Clustering. In *International Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:232075682

[9] Hu Ding, Yu Liu, Lingxiao Huang, and Jian Li. 2016. K-means clustering with distributed dimensions. In *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48* (New York, NY, USA) *(ICML'16)*. JMLR.org, 1339–1348.

[10] Wei Feng, Zhenwei Wu, Qianqian Wang, Bo Dong, Zhiqiang Tao, and Quanxue Gao. 2024. Efficient Federated Multi-View Clustering with Integrated Matrix Factorization and K-Means. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, Kate Larson (Ed.). International Joint Conferences on Artificial Intelligence Organization, 3971–3979.

[11] Fangcheng Fu, Xupeng Miao, Jiawei Jiang, Huanran Xue, and Bin Cui. 2022. Towards communication-efficient vertical federated learning training via cache-enabled local updates. *Proc. VLDB Endow.* 15, 10 (June 2022), 2111–2120. doi:10.14778/3547305.3547316

[12] Fangcheng Fu, Yingxia Shao, Lele Yu, Jiawei Jiang, Huanran Xue, Yangyu Tao, and Bin Cui. 2021. Vf2boost: Very fast vertical federated gradient boosting for cross-enterprise learning. In *Proceedings of the 2021 International Conference on Management of Data*. 563–576.

[13] Quanxue Gao, Fangfang Li, Qianqian Wang, Xinbo Gao, and Dacheng Tao. 2025. Manifold Based Multi-View <italic>K</italic>-Means. *IEEE Trans. Pattern Anal. Mach. Intell.* 47, 4 (April 2025), 3175–3182.

[14] Cristiano Gratton, Naveen KD Venkategowda, Reza Arablouei, and Stefan Werner. 2018. Distributed ridge regression with feature partitioning. In *2018 52nd Asilomar Conference on Signals, Systems, and Computers*. IEEE, 1423–1427.

[15] Bin Gu, An Xu, Zhouyuan Huo, Cheng Deng, and Heng Huang. 2021. Privacy-preserving asynchronous vertical federated learning algorithms for multiparty collaborative learning. *IEEE transactions on neural networks and learning systems* 33, 11 (2021), 6103–6115.

[16] Patrick Holzer, Tania Jacob, and Shubham Kavane. 2023. Dynamically Weighted Federated k-Means. *ArXiv* abs/2310.14858 (2023).

[17] Lingxiao Huang, Zhize Li, Jialin Sun, and Haoyu Zhao. 2022. Coresets for Vertical Federated Learning: Regularized Linear Regression and K-Means Clustering, Vol. 35. Curran Associates, Inc., 29566–29581.

[18] Shudong Huang, Ivor W. Tsang, Zenglin Xu, and Jiancheng Lv. 2022. Measuring Diversity in Graph Learning: A Unified Framework for Structured Multi-View Clustering. *IEEE Transactions on Knowledge and Data Engineering* 34, 12 (2022), 5869–5883. doi:10.1109/TKDE.2021.3068461

[19] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. 2021. Advances and open problems in federated learning. *Foundations and trends® in machine learning* 14, 1–2 (2021), 1–210.

[20] Zhao Kang, Guoxin Shi, Shudong Huang, Wenyu Chen, Xiaorong Pu, Joey Tianyi Zhou, and Zenglin Xu. 2020. Multi-graph fusion for multi-view spectral clustering. *Knowledge-Based Systems* 189 (2020), 105102.

[21] Anran Li, Hongyi Peng, Lan Zhang, Jiahui Huang, Qing Guo, Han Yu, and Yang Liu. 2023. Fedsdg-fs: Efficient and secure feature selection for vertical federated learning. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 1–10.

[22] Chao Li, Shifei Ding, Xiao Xu, Lili Guo, Ling Ding, and Xindong Wu. 2025. Vertical Federated Density Peaks Clustering Under Nonlinear Mapping. *IEEE Transactions on Knowledge and Data Engineering* 37, 2 (2025), 1004–1017. doi:10.1109/TKDE.2024.3487534

[23] Feijiang Li, Xin Liu, Jieting Wang, and Yuhua Qian. 2025. MCSS: Discovering Consistently Determined Relation in Multi-View Clustering Based on Sample's Stability. *ACM Trans. Knowl. Discov. Data* 20, 1, Article 2 (Nov. 2025), 28 pages.

[24] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems* 2 (2020), 429–450.

[25] Yingming Li, Ming Yang, and Zhongfei Zhang. 2018. A survey of multi-view representation learning. *IEEE transactions on knowledge and data engineering* 31, 10 (2018), 1863–1883.

[26] Zhenglai Li, Chang Tang, Xinwang Liu, Xiao Zheng, Wei Zhang, and En Zhu. 2022. Consensus Graph Learning for Multi-View Clustering. *IEEE Transactions on Multimedia* 24 (2022), 2461–2472. doi:10.1109/TMM.2021.3081930

[27] Ke Liang, Yue Liu, Sihang Zhou, Wenxuan Tu, Yi Wen, Xihong Yang, Xiangjun Dong, and Xinwang Liu. 2023. Knowledge graph contrastive learning based on relation-symmetrical structure. *IEEE Transactions on Knowledge and Data Engineering* 36, 1 (2023), 226–238.

[28] Ke Liang, Lingyuan Meng, Hao Li, Jun Wang, Long Lan, Miaomiao Li, Xinwang Liu, and Huaimin Wang. 2025. From Concrete to Abstract: Multi-view Clustering on Relational Knowledge. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025), 1–18. doi:10.1109/TPAMI.2025.3582689

[29] Suyuan Liu, Siwei Wang, Pei Zhang, Kai Xu, Xinwang Liu, Changwang Zhang, and Feng Gao. 2022. Efficient One-Pass Multi-View Subspace Clustering with Consensus Anchors. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 7 (Jun. 2022), 7576–7584.

[30] Suyuan Liu, Hao Yu, Hao Tan, Ke Liang, Siwei Wang, Shengju Yu, En Zhu, and Xinwang Liu. 2025. Efficient Federated Incomplete Multi-View Clustering. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*, Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.). PMLR, 40100–40114.

[31] Yang Liu, Yan Kang, Tianyuan Zou, Yanhong Pu, Yuanqin He, Xiaozhou Ye, Ye Ouyang, Ya-Qin Zhang, and Qiang Yang. 2024. Vertical Federated Learning: Concepts, Advances, and Challenges. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (July 2024), 3615–3634.

[32] Shirui Luo, Changqing Zhang, Wei Zhang, and Xiaochun Cao. 2018. Consistent and Specific Multi-View Subspace Clustering. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence*. AAAI Press, 8 pages.

[33] Juncheng Lv, Zhao Kang, Boyu Wang, Luping Ji, and Zenglin Xu. 2021. Multi-view subspace clustering via partition fusion. *Information Sciences* 560 (2021), 410–423.

[34] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the Artificial intelligence and statistics*. PMLR, 1273–1282.

[35] Jingwei Sun, Ziyue Xu, Dong Yang, Vishwesh Nath, Wenqi Li, Can Zhao, Daguang Xu, Yiran Chen, and Holger R Roth. 2023. Communication-efficient vertical federated learning with limited overlapping samples. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5203–5212.

[36] Shiliang Sun. 2013. A survey of multi-view machine learning. *Neural computing and applications* 23 (2013), 2031–2038.

[37] Shiliang Sun, Liang Mao, Ziang Dong, and Lidan Wu. 2019. *Multiview machine learning*. Springer.

[38] Chen Tianyi, Jin Xiao, Sun Yuejiao, and Yin Wotao. 2020. VAFL: a Method of Vertical Asynchronous Federated Learning. *ArXiv* abs/2007.06081 (2020).

[39] Feng Wang, M. Cenk Gursoy, and Senem Velipasalar. 2024. Feature-Based Federated Transfer Learning: Communication Efficiency, Robustness and Privacy. *IEEE Transactions on Machine Learning in Communications and Networking* 2 (2024), 823–840.

[40] Qianqian Wang, Zhiqiang Tao, Quanxue Gao, and Licheng Jiao. 2024. Multi-view subspace clustering via structured multi-pathway network. *IEEE Transactions on*

Neural Networks and Learning Systems 35, 5 (2024), 7244–7250.
[41] Guo Wei, Zhuang Fuzhen, Zhang Xiao, Tong Yiqi, and Dong Jin. 2024. A Comprehensive Survey of Federated Transfer Learning: Challenges, Methods and Applications. *CoRR* abs/2403.01387 (2024).
[42] Yu Zang, Zhe Xue, Shilong Ou, Lingyang Chu, Junping Du, and Yunfei Long. 2024. Efficient asynchronous federated learning with prospective momentum aggregation and fine-grained correction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 16642–16650.
[43] Qingsong Zhang, Bin Gu, Cheng Deng, Songxiang Gu, Liefeng Bo, Jian Pei, and Heng Huang. 2021. AsySQN: Faster vertical federated learning algorithms with better computation resource utilization. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 3917–3927.
[44] Qingsong Zhang, Bin Gu, Cheng Deng, and Heng Huang. 2021. Secure bilevel asynchronous vertical federated learning with backward updating. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 10896–10904.
[45] Xunzheng Zhang, Alex Mavromatis, Antonis Vafeas, Reza Nejabati, and Dimitra Simeonidou. 2023. Federated Feature Selection for Horizontal Federated Learning in IoT Networks. *IEEE Internet of Things Journal* 10, 11 (2023), 10095–10112.
[46] Yunfan Zhang, Yiqun Zhang, Yang Lu, Mengke Li, Xi Chen, and Yiu-ming Cheung. 2025. Asynchronous Federated Clustering with Unknown Number of Clusters. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 21 (Apr. 2025), 22695–22703. doi:10.1609/aaai.v39i21.34429

A Appendix

A.1 Proof of Theorem 3.1

Before the proof of Theorem 3.1, we first introduce a theorem as follows.

Theorem A.1. *Let $A \in \mathbb{R}^{d\times d}$ be an invertible matrix with singular value decomposition $A = U\Sigma V^T$, where U and V are orthogonal matrices, and Σ is a diagonal matrix containing the singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_d$ of A. Let σ_{max} and σ_{min} be the largest and smallest singular values of A, respectively. Then, for any vectors $x_i, x_j \in \mathbb{R}^d$, the Euclidean distance between the transformed vectors x_iA and x_jA satisfies the following inequality:*

$$\sigma^2_{min}(A)\|x_i - x_j\|_2^2 \leq \|x_iA - x_iA\|_2^2 \leq \sigma^2_{max}(A)\|x_i - x_j\|_2^2. \quad (21)$$

Proof. We begin by expressing the distance between the transformed vectors x_iA and x_jA using the SVD of A:

$$\|(x_i - x_j)A\|_2^2 = \|(x_i - x_j)U\Sigma V^T\|_2^2 \quad (22)$$

Since V is an orthogonal matrix, it preserves the Euclidean norm:

$$\|(x_i - x_j)U\Sigma V^T\|_2^2 = \|(x_i - x_j)U\Sigma\|_2^2. \quad (23)$$

Let $z = (x_i - x_j)U$. Because U is orthogonal, the norm of z is equal to the norm of $x_i - x_j$:

$$\|z\|_2^2 = \|(x_i - x_j)U\|_2^2 = \|x_i - x_j\|_2^2. \quad (24)$$

Thus, we have:

$$\|(x_i - x_j)U\Sigma\|_2^2 = \|z\Sigma\|_2^2. \quad (25)$$

Since Σ is a diagonal matrix with singular values $\sigma_1, \sigma_2, \ldots, \sigma_d$ on the diagonal, it scales each component of z:

$$\|\Sigma z\|_2^2 = \sum_{k=1}^{d}(\sigma_k z_k)^2 \quad (26)$$

The largest and smallest singular values of A are $\sigma^2_{\max}(A)$, and $\sigma^2_{\min}(A)$, respectively. Therefore, we can bound the norm of Σz as follows:

$$\sigma^2_{\min}(A)\|z\|_2^2 \leq \|\Sigma z\|_2^2 \leq \sigma^2_{\max}(A)\|z\|_2^2 \quad (27)$$

Substituting $\|z\|_2^2 = \|x_i - x_j\|_2^2$, we obtain:

$$\sigma^2_{\min}(A)\|x_i - x_j\|_2^2 \leq \|x_iA - x_iA\|_2^2 \leq \sigma^2_{\max}(A)\|x_i - x_j\|_2^2, \quad (28)$$

which completes the proof. □

THEOREM 3.1. *Let $\mathcal{I}'_{opt}$ be the optimal partition on the transformed data $D' = DA$, where $A \in \mathbb{R}^{d\times d}$, suppose there exists a γ ($\gamma > 1$) approximation K-means algorithm for D' which can obtain a partition $D'_1 \cup \cdots \cup D'_K$ (the corresponding index partition is $\mathcal{I}' = I'_1 \cup \cdots \cup I'_K$) satisfying:*

$$\Delta^2(x'; I') \leq \gamma\Delta^2(x'; I'_{opt}). \quad (29)$$

then the k-means cost of obtained partition on the original data satisfies:

$$\Delta^2(x; I') \leq \gamma\kappa(AA^T)\Delta^2(x; I_{opt}). \quad (30)$$

where $\kappa(AA^T) = \frac{\sigma^2_{max}(AA^T)}{\sigma^2_{min}(AA^T)}$ is the condition number of AA^T, and σ is singular value.

Proof. The core idea of this proof lies in amplifying intra-cluster distances between sample points through the application of Theorem A.1. Specifically, we have:

$$\begin{aligned}
\Delta^2(x; I') &= \sum_{k=1}^{K'}\Delta^2(x; I'_k) && (31)\\
&= \sum_{k=1}^{K'}\frac{1}{2n'_k}\sum_{i\in I'_k, j\in I'_k}\|x_i - x_j\|_2^2 && (32)\\
&\leq \sum_{k=1}^{K'}\frac{1}{2n'_k}\sum_{i\in I'_k, j\in I'_k}\frac{1}{\sigma^2_{\min}(A)}\|x_iA - x_jA\|_2^2 && (33)\\
&= \frac{1}{\sigma_{\min}(AA^T)}\sum_{k=1}^{K'}\Delta^2(x'; I'_k) && (34)\\
&\leq \frac{\gamma}{\sigma_{\min}(AA^T)}\sum_{k=1}^{K'}\Delta^2(x'; I'_{opt,k}) && (35)\\
&= \frac{\gamma}{\sigma_{\min}(AA^T)}\Delta^2(x'; I'_{opt}), && (36)
\end{aligned}$$

where the last inequality is by the γ-optimality of partition D'.

Furthermore, by the optimality of D'_{opt} on D', we have,

$$\Delta^2(x'; I'_{opt}) \le \Delta^2(x'; I_{opt}) \tag{37}$$

$$= \sum_{k=1}^{K_{opt}} \Delta^2(x'; I_{opt,k}) \tag{38}$$

$$= \sum_{k=1}^{K_{opt}} \frac{1}{2n_{opt,k}} \sum_{i \in I_{opt,k}, j \in I_{opt,k}} \|x'_i - x'_j\|_2^2 \tag{39}$$

$$\le \sum_{k=1}^{K_{opt}} \frac{1}{2n_{opt,k}} \sum_{i \in I_{opt,k}, j \in I_{opt,k}} \sigma_{\max}(AA^T)\|x_i - x_j\|_2^2 \tag{40}$$

$$= \sigma_{\max}(AA^T)\Delta^2(x; I_{opt}), \tag{41}$$

where the last inequality is by Theorem A.1.

Combining the above results, we have can obtain the final result. □

A.2 Discussion of the rationality of D_i^{new} and the uploaded information u_i

According to Equation (5) in the main text, the data projection is essentially obtained by performing feature-wise sampling with replacement and then averaging the results. For the i-th sampling, the sampling process can be represented by a sampling matrix

$$A^i = [\frac{c_1^i}{d}, \frac{c_2^i}{d}, \ldots, \frac{c_d^i}{d}]^T,$$

where d is the number of features and c_j^i is selection count of the j-th feature.

Performing this sampling d times yields a transformation matrix, which is

$$A = [A^1, A^2, \ldots, A^d].$$

For the data set in client i D_i, the D_i^{new} is obtained by the transformation of

$$D_i^{new} = [D_i I_d, D_i A]^T,$$

where I_d is the $d \times d$ identity matrix.

It is easy to obtain that $\kappa(I_d {I_d}^T) = 1$. For the matrix A, since it is generated through random sampling with replacement, the features that are not selected in each sampling round appear randomly, and the number of times a feature is selected is also random. As a result, there is a high probability that A is full-rank. Given that A is full-rank, $\kappa(AA^T) \approx 1$. Then D_i^{new} is the projected data that preserves the k-means loss, which shows the rationality.

To further compress the information, we perform clustering on D_i^{new} to obtain the cluster centroids u_i, which are then uploaded to the server.

These uploaded centroids carry meaningful information, as the pairwise distances among them are related to the k-means loss.

Let $D = \{x_1, \ldots, x_n\}$ be the input data with a number of d features and $\mathcal{I} = \{I_1, \ldots, I_K\}$ forms a disjoint partition of the complete set $\{1, \ldots, n\}$, where I_k is the index set of samples in cluster k. That is, the k-th cluster is $D_k = \{x_i \mid i \in I_k\}$ and the complete clustering is $D = \bigcup_{k=1}^K D_k$. Let $\mathrm{ctr}(S)$ denote the center of mass of S. Then the center of cluster is $c_k = \mathrm{ctr}(D_k)$ and the total center is $c = \mathrm{ctr}(X)$. The K-means cost of the partition is,

$$\Delta^2(x; I) = \sum_{k=1}^K \sum_{x \in X_k} \|x - c_k\|_2^2 = \sum_{k=1}^K \Delta^2(x; I_k). \tag{42}$$

Through the addition and subtraction of the term c_k and simple calculations, for every x_i, we have,

$$\sum_{x_j \in X_k} \|x_i - x_j\|_2^2 = \Delta^2(x; I_k) + n_k\|x_i - c_k\|_2^2. \tag{43}$$

Then, we can derive the relationship between pairwise distances and the loss:

$$\sum_{x_i \in X_k, x_j \in X_k} \|x_i - x_j\|_2^2 = 2n_k\Delta^2(x; I_k). \tag{44}$$